Centre universitari adscrit a la

Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Memory

Volume I

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

April 2026

Contents

List of Figures V

List of Tables VII

Glossary IX

Dedication and Acknowledgements 1

1 Introduction 3
1.1 Problem Statement and Educational Gap 3
1.2 Proposed Solution 4

2 State of the Art 5
2.1 Context 5
2.2 Existing Platforms Overview 6
2.3 Security Frameworks and Methodologies 7
2.3.1 PTES – Penetration Testing Execution Standard . . . 7
2.3.2 OWASP Top 10 7
2.3.3 CIS Critical Controls 7
2.3.4 NIST Cybersecurity Framework 8
2.4 Prior Academic Research 8
2.5 Gap Analysis: What Existing Solutions Miss 8
2.6 The Need for EVE-NG Based Labs 9

3 Objectives and Scope 11
3.1 Specific Objectives (Measurable with KPIs) 11
3.1.1 OE1: Design and Implement EVE-NG Labs 11
3.1.2 OE2: Develop Automation Scripts 11
3.1.3 OE3: Develop Structured Teaching Material 12
3.1.4 OE4: Validate Usability and Educational Effectiveness 12
3.2 Definition of the Client and Final User 13
3.2.1 Principal Client 13
3.2.2 Primary End Users 13
3.3 Scope Boundaries 13
3.4 KPIs and Performance Indicators 15

4 Methodology 17
4.1 Research and Development Phases 17
4.1.1 Phase 0: Preparation and Infrastructure (July–November 2025) 17
4.1.2 Phase 1: Preliminary Project Proposal and Conceptual Design (October 2025–January 2026) 17
4.1.3 Phase 2: Implementation and Interim Validation (January–March 2026) 18

4.1.4 Phase 3: Completion and Final Validation (March–May 2026) . . . 18
4.2 Information Search Strategies . . . 18
4.3 Development Methodology . . . 19
4.4 Platform Selection Rationale . . . 19
4.5 Validation and Success Criteria . . . 19
4.6 Risk Management . . . 20

5 Functional and Non-Functional Requirements 21
5.1 Functional Requirements . . . 21
5.1.1 RF1: Design and Implement EVE-NG Topologies . . . 21
5.1.2 RF2: Implement Vulnerabilities and Attack Scenarios 21
5.1.3 RF3: Create Structured Assessment Materials . . . 22
5.2 Technological Requirements . . . 23
5.2.1 TR1: Virtualisation Infrastructure . . . 23
5.2.2 TR2: Security Tools and Penetration Testing Framework . . . 23
5.2.3 TR3: Scripting and Automation . . . 23
5.3 Non-Functional Requirements . . . 24
5.4 Requirements Traceability Matrix . . . 24

6 Feasibility Study 27
6.1 Gantt Chart and Task Definition . . . 27
6.1.1 Critical Path . . . 27
6.2 Budget . . . 28
6.2.1 Human Resources . . . 28
6.2.2 Hardware and Software . . . 29
6.2.3 Other Costs . . . 29
6.2.4 Total Budget . . . 30
6.3 Feasibility Analysis . . . 30
6.3.1 Technical Feasibility . . . 30
6.3.2 Economic Feasibility . . . 31
6.3.3 Temporal Feasibility . . . 31
6.3.4 Environmental Feasibility . . . 32
6.3.5 Legal Aspects . . . 33
6.3.6 Gender and Diversity Perspective . . . 34

7 Development and Implementation 37
7.1 Platform Setup . . . 37
7.2 Image Management . . . 38
7.2.1 Image Selection . . . 38
7.2.2 Image Organisation in EVE-NG . . . 39
7.2.3 ISO Upload Automation . . . 39
7.3 EVE-NG Lab Management Interface . . . 40
7.4 Infrastructure Problems . . . 41
7.4.1 EVE-NG on VMware . . . 41
7.4.2 Node Configuration Persistence . . . 41
7.4.3 LVM Volume Group Naming Conflict . . . 42

7.4.4 Memory Capacity Constraint 42
7.5 Lab Topologies 43
7.6 Lab 1: Reconnaissance and Enumeration 43
7.6.1 Problems Encountered and Solutions 44
7.7 Lab 2: Web Application Vulnerabilities — SYNAPSE Intelligence Portal 46
7.7.1 Problems Encountered and Solutions 47
7.8 Lab 3: Incident Response and Log Forensics — HELIX Systems 49
7.8.1 Problems Encountered and Solutions 50
7.9 Pilot Validation 51
7.9.1 Pilot Design 51
7.9.2 Quantitative Results 52
7.9.3 Qualitative Analysis 53
7.9.4 KPI Compliance 54
7.9.5 Identified Improvements 54

8 Conclusions **57**
8.1 Summary of Achievements 57
8.2 Objectives Evaluation 58
8.3 Lessons Learned 58
8.4 Limitations and Future Work 60
8.4.1 Current Limitations 60
8.4.2 Directions for Future Work 60
8.5 Personal Reflection 61

Bibliography **63**

List of Figures

7.1 EVE-NG File Manager showing the lab directory structure, import controls, and topology thumbnails. 40

List of Tables

2.1 Comparative Analysis: Existing Cybersecurity Training Platforms 6

3.1 Key performance indicators for the lab teaching package 15

4.1 Key project risks and mitigation strategies 20

5.1 Non-functional requirements summary 24
5.2 Requirements traceability 24

6.1 Distribution of tasks by category 27
6.2 Critical path tasks 28
6.3 Human resources cost (junior engineer rate, 15 €/h, theoretical) 28
6.4 Hardware costs 29
6.5 Software and license costs (all free/open-source for educational use) 29
6.6 Additional operational costs 29
6.7 Complete project budget 30
6.8 Required technical competencies 30
6.9 Risk assessment 31
6.10 Cost comparison 31
6.11 CO^2 comparison 32

7.1 Overview of the four EVE-NG lab topologies designed for the course. 43
7.2 Lab 1 IP addressing scheme 44
7.3 Lab 1 techniques and framework mapping 44
7.4 Lab 1 flags and acquisition techniques 46
7.5 Lab 2 IP addressing scheme 47
7.6 Lab 2 OWASP Top 10 vulnerability chain 47
7.7 Lab 3 IP addressing scheme 49
7.8 Lab 3 MITRE ATT&CK technique coverage 50
7.9 Pilot study results — mean scores (1–5 Likert) and resolution times 52
7.10 OE4 KPI compliance after pilot study 54

8.1 Specific objectives evaluation at submission 58

Glossary

API REST

Application Programming Interface following the *Representational State Transfer* (REST) architectural style, using HTTP to enable communication between systems and supporting standard resource operations.

Burp Suite

An integrated web application security testing platform enabling the interception, modification, and analysis of HTTP/HTTPS traffic. Used for web vulnerability exercises in Lab 2.

CIS Critical Controls

A consensus-based set of 18 prioritised security controls from the Center for Internet Security, representing best practices for defending against the most prevalent attack vectors.

Cross-Site Scripting (XSS)

A vulnerability enabling the injection of malicious client-side scripts into web pages viewed by other users, thereby compromising session tokens or exfiltrating sensitive data.

CTF (Capture The Flag)

A cybersecurity competition format in which participants solve challenges to find hidden strings (flags) that prove successful exploitation. The lab progression model used throughout this thesis is based on a linear CTF chain.

CVSS (Common Vulnerability Scoring System)

A standardised framework for quantifying vulnerability severity through a numerical score—ranging from 0 to 10—that reflects exploitability and potential impact on affected systems.

DMZ (Demilitarized Zone)

A neutral security segment segregating an organisation's internal network from the public Internet, hosting externally-accessible

services whilst protecting critical internal resources from direct exposure.

DNS (Domain Name System)

The hierarchical distributed naming system that resolves human-readable hostnames to IP addresses. Used in Lab 1 as the entry point for reconnaissance: the custom hostname `lab1` is resolved via the pfSense Unbound DNS resolver.

Enumeration

A methodical process of systematically identifying and documenting network resources, active services, user accounts, and group memberships—typically following network scanning. Essential for pinpointing viable attack targets.

Ethical Hacking

The authorised and legal practice of proactively exploiting security vulnerabilities with explicit permission, conducted within a framework of ethical and legal responsibility. Conceptual foundation of this educational endeavour.

EVE-NG

Emulated Virtual Environment — Next Generation; a network emulation platform enabling the construction of complex multi-node topologies with diverse virtual machines and networking devices. Foundational platform of this thesis.

Exploit

A piece of code, technique, or procedure that strategically targets a specific vulnerability to achieve unauthorised access or execute arbitrary code on a target system.

HomeLab

A personal infrastructure environment combining server hardware and virtualisation software for educational purposes, experimentation, and professional development. Serves as the physical foundation for this thesis.

Incident Response

A structured methodology for detecting, containing, eradicating, and recovering from cybersecurity incidents. Lab 3 of this thesis simulates an active incident at HELIX Systems, requiring students to follow an IR workflow aligned with NIST SP 800-61.

Kali Linux

A Debian-based Linux distribution tailored for penetration testing and security auditing, providing hundreds of pre-installed tools for scanning, exploitation, and analysis. Widely used in industry and referenced in certification curricula such as OSCP.

Lab Topology

The architectural arrangement of virtual machines, network segments, and services within an educational laboratory environment, designed to simulate realistic penetration testing scenarios.

Lateral Movement

A post-exploitation technique in which an attacker uses a compromised system as a pivot point to access other systems within the same network, typically with the aim of reaching higher-value targets or sensitive data. Demonstrated in Lab 3 via SSH credential reuse from Server-Web to Server-DB.

Local Privilege Escalation

A privilege escalation technique executed locally on an already-compromised machine, exploiting vulnerabilities in the operating system kernel or misconfigured service permissions.

Metasploit Framework

An open-source exploitation framework facilitating the development, customisation, and execution of exploits against target systems. Industry standard for professional penetration testing engagements.

NAT (Network Address Translation)

A technique in which a router or firewall modifies the source or destination IP addresses of packets in transit, enabling multiple

hosts on a private network to share a single public address and allowing controlled exposure of internal services.

Network Scanning

The process of discovering active hosts, open ports, and running services across a network using specialised tools such as Nmap. The foundational step of offensive reconnaissance.

Network Topology

The physical and logical arrangement of network components—including computers, servers, networking devices, and interconnections. Critical for designing realistic laboratory environments.

nginx

A high-performance open-source web server and reverse proxy. Used in Lab 1 as the HTTP server on the target node, serving the PEBCAK Corp portal.

NIST Cybersecurity Framework

A standards-based framework from the National Institute of Standards and Technology for managing cybersecurity risk, organised into five functional domains: Identify, Protect, Detect, Respond, and Recover.

Nmap

An open-source network scanning tool that discovers active hosts, enumerates open ports, identifies running services, and fingerprints operating systems. Essential during the reconnaissance phase.

OWASP

Open Web Application Security Project; an international non-profit community dedicated to advancing web application security. Publishes the industry-standard Top 10 catalogue of critical vulnerabilities and comprehensive testing methodologies.

OWASP ZAP

An open-source web application scanner offering both automated scanning and interactive testing workflows. A freely available alternative to commercial web security tools.

Packet Capture

The process of intercepting and recording data packets traversing a network, enabling the analysis of protocols, unencrypted communications, and anomalous traffic patterns.

Parrot OS

A Debian-based Linux distribution oriented towards security testing, privacy, and development. Used as the operating system of the attacker node in the lab environment of this thesis.

Payload

The component or data transmitted as part of an attack vector, encapsulating the malicious code or instructions the attacker intends to execute within the compromised system.

Pentesting

Penetration testing; an authorised, controlled simulation of cyberattacks designed to identify security vulnerabilities across systems, networks, and applications. Fundamental component of this thesis.

pfSense

An open-source firewall and router platform based on FreeBSD, distributed by Netgate. Used in this project as the perimeter firewall, providing NAT, port forwarding, and DNS resolution for the lab environment.

Privilege Escalation

A technique enabling a user with limited permissions to obtain elevated privileges—such as administrator or root access—thereby executing previously restricted operations. Central focus of Lab 4.

Proxmox VE

An open-source server virtualisation platform based on Debian Linux, supporting both KVM virtual machines and LXC containers. Used in this project as the hypervisor hosting the EVE-NG virtual machine.

Reconnaissance

The initial phase of a penetration test during which the attacker gathers intelligence about the target—encompassing systems, users, networks, and services—typically through passive techniques without direct system access. Primary focus of Lab 1 of this thesis.

Root Access

Maximum-level administrative access on Unix/Linux systems, equivalent to Administrator privileges on Windows. A typical objective in attack chains and the culmination of exploitation phases.

SQL Injection

A critical vulnerability that permits an attacker to insert malicious SQL code into database queries, enabling unauthorised data access, modification, or deletion. Primary focus of Lab 2.

SSH (Secure Shell)

A cryptographic network protocol providing secure command-line access, file transfer, and tunnelling over an unsecured network. Used in Lab 1 as the final step of the student workflow to access the target server and retrieve the flag.

TFG (Bachelor's Thesis)

A mandatory capstone project in Spanish university degree programmes, demonstrating the comprehensive knowledge and competencies acquired throughout the academic programme.

Virtual Machine

A software-based emulation of a complete computer system running within a physical host, enabling the concurrent execution of multiple operating systems and the creation of isolated, reproducible laboratory environments.

Vulnerability

A flaw or weakness inherent in a system, application, or infrastructure that can be exploited by an attacker to gain unautho-

rised access, inflict damage, or compromise data integrity and confidentiality.

Vulnerability Assessment

A systematic process of identifying, quantifying, and prioritising security vulnerabilities across systems, networks, and applications. Typically precedes comprehensive penetration testing engagements.

VyOS

An open-source network operating system based on Debian, providing routing, firewalling, and VPN functionality. Used in this project as the internal Layer 3 router connecting the lab network segments.

Wireshark

An open-source network protocol analyser that captures and examines network traffic in real time. Particularly relevant for network traffic analysis exercises and referenced in Lab 2 and Lab 3 of this thesis.

Dedication and Acknowledgements

Dedication

This work is dedicated to all students and professionals interested in cybersecurity, particularly in ethical pentesting. We hope that the labs and scenarios presented in this bachelor's thesis will be useful for their training and professional development.

Acknowledgements

Institutional Acknowledgements

I would like to express my sincere gratitude to my tutor, Pere Vidiella i Catalan, for his guidance, trust, and constructive feedback throughout this project. I am also deeply grateful to the faculty and staff of the Computer Engineering program for providing the academic framework, resources, and supportive environment that made this work possible. Their dedication to excellence in teaching and mentoring has been instrumental in shaping both the technical scope and the academic rigor of this thesis.

Personal Acknowledgements

This thesis is dedicated, above all, to my grandmother, who passed away just before I began my degree—shortly after my university admission and the day before my English examination. Choosing to write this thesis in English is, in part, a meaningful way to close a chapter that marked the beginning of my university journey without the person who meant the most to me and who, in many ways, was like a mother to me.

I am also profoundly grateful to my friends, my partner, and my partner's family for their unwavering support and encouragement, especially during the most difficult moments of my studies, including a

period when I faced a serious health challenge that could have left me paralyzed. Their presence, care, and belief in me have been essential to completing this work.

Use of Artificial Intelligence Tools

During the development of this thesis, complementary AI tools were used exclusively for review and error-detection purposes: a locally-hosted language model (LM Studio) and an online assistant (Claude, Anthropic). These tools were not used to generate any academic content; all analysis, design, implementation, and writing are the author's own work.

Personal Reflection

The completion of this Bachelor's Thesis has been an enriching experience that has allowed me to deepen my knowledge of cybersecurity and develop technical and project management skills. I have learned the importance of practice in learning this discipline and how virtualized environments can facilitate this process. This project has motivated me to continue exploring the field of cybersecurity and to contribute to the training of future professionals in this critical area.

1 Introduction

1.1 Problem Statement and Educational Gap

Cybersecurity education faces a persistent challenge: the gap between theoretical knowledge of IT systems and the practical understanding of how security failures manifest in real, integrated environments. Students in computer engineering programmes learn fundamental concepts in networks, operating systems, databases, and software architecture — yet they rarely see how misconfigurations and design flaws in these interconnected components translate into exploitable vulnerabilities. Without exposure to multi-component environments that simulate real-world systems, graduates enter the workforce lacking the diagnostic reasoning and hands-on skills that professional security roles demand.

This gap has become more acute with the widespread adoption of AI-assisted development tools. Code generation assistants such as GitHub Copilot and ChatGPT allow developers to produce functional code rapidly — authentication logic, input validation routines, database queries — without necessarily understanding the security implications of what is being generated. Vulnerabilities that a security-aware developer would immediately recognise — SQL injection sinks, broken access control, unsafe deserialization — are routinely introduced through AI-generated code integrated without critical review. The result is a growing population of practitioners who can build software but cannot evaluate its attack surface. Practical security education that requires students to actively exploit and reason about these vulnerabilities — rather than generate or copy solutions — is therefore more urgent than ever.

Within the TecnoCampus Bachelor's Degree in Computer Engineering (Management and Information Systems track), the *Introduction to Cybersecurity* course currently lacks a structured, reusable, and automatable lab environment that:

- Aligns pentesting scenarios with specific degree learning out-

comes

- Exposes students to vulnerabilities within integrated, multi-component systems rather than isolated challenge-based scenarios
- Provides reproducible environments for consistent assessment across cohorts
- Reduces teacher workload through automation and structured documentation
- Remains cost-effective and institution-independent, deployable on open-source infrastructure without vendor lock-in

1.2 Proposed Solution

This project proposes a reusable penetration testing lab package built on EVE-NG [1] and deployed on self-hosted infrastructure, designed specifically for the *Introduction to Cybersecurity* course at Tecno-Campus. The package comprises four progressive lab scenarios: network reconnaissance and enumeration, web application vulnerabilities, incident response and log forensics, and privilege escalation.

Each lab is self-contained: it includes a network topology file (`.unl`), a student lab guide, an teacher resolution guide with assessment rubric, and can have supporting automation scripts. Progress is structured around a flag-based model — students obtain verifiable tokens at each milestone only by completing the required technical steps, which prevents superficial shortcuts and ensures that understanding the attack process is the only path to completion. The entire package is open-source, vendor-independent, and deployable on any institution running a compatible hypervisor. Full teacher references and deployment details are available in Volume II (Appendices) and at the project documentation site.

2 State of the Art

2.1 Context

The Centre Universitari TecnoCampus [2] offers applied science degrees oriented towards professional practice in technology, business, creative industries, and health. Affiliated with the Universitat Pompeu Fabra, it carries that university's quality seal. Established in 2023 through the merger of three schools — the Escola Superior Politecnica (ESUPT), the Escola Superior de Ciencies Socials i de l'Empresa (ESCSET), and the Escola Superior de Ciencies de la Salut (ESCST) — it is located at the technology park of the same name in Mataro. This setting combines academic infrastructure with an entrepreneurial ecosystem that supports student involvement in innovation and industry-facing projects.

The primary beneficiary of this project is the Bachelor's Degree in Computer Engineering (Management and Information Systems track). Students in this programme build foundational skills in networks, operating systems, databases, and software engineering — all of which are directly relevant to security practice. The *Introduction to Cybersecurity* course is a natural integration point for these skills, yet it currently lacks a dedicated hands-on environment.

Three perspectives illustrate the gap this project addresses:

Institutional perspective. The absence of dedicated cybersecurity lab infrastructure limits the institution's capacity to offer competitive, hands-on security training aligned with degree learning outcomes. Commercial alternatives such as HackTheBox and TryHackMe are externally hosted, non-customisable, and not designed to map to specific academic curricula or assessment rubrics.

Student perspective. Students encounter security concepts as isolated theoretical constructs rather than as failures observable in real-

istic, integrated environments. Without exposure to multi-component systems — firewalls, routers, servers, and applications working together — they cannot develop the diagnostic reasoning required in professional security roles.

Teacher perspective. Designing and maintaining practical security exercises is resource-intensive. The absence of reusable, automatable lab environments, teachers must improvise or rely on third-party platforms, which reduces pedagogical control and limits reproducibility across cohorts.

2.2 Existing Platforms Overview

The major platforms used for cybersecurity training share a common limitation: they are designed for independent learners or certification preparation rather than curriculum-aligned institutional deployment. Table 2.1 summarises their key characteristics. Platforms such as HackTheBox [3] and TryHackMe [4] are externally hosted and cannot be deployed on institutional infrastructure. SEED Labs [5] and GOAD [6] support local deployment but cover only isolated concepts or narrow domains. VulnHub [7] and Hack4u [8] lack automation and assessment integration. Cisco Networking Academy [9] provides formal structure but is tightly coupled to vendor technology and limited to network simulation.

Platform	Approach	Deploy	Curriculum	Auto	Network
HackTheBox	CTF	No	Minimal	No	Isolated
TryHackMe	Gamified	No	Structured	No	Limited
Cisco Academy	Academic	Yes (PT)	Strong	Limited	Net-focused
SEED Labs	Conceptual	Yes (Docker)	Concept-based	Minimal	Isolated
GOAD	AD-focused	Yes (Terraform)	None	Moderate	AD Domain
VulnHub	Community	Yes	None	No	Isolated
Hack4u Academy	Methodology	External	Strong	Minimal	Limited

Table 2.1: Comparative Analysis: Existing Cybersecurity Training Platforms

Full individual platform descriptions, including detailed discussion of each platform's pedagogical approach, deployment model, and specific limitations, are provided in the project online documentation.

2.3 Security Frameworks and Methodologies

Several authoritative frameworks inform cybersecurity education and guide both attack methodologies and defensive best practices.

2.3.1 PTES – Penetration Testing Execution Standard

The Penetration Testing Execution Standard (PTES) [10] defines a complete methodology for professional penetration testing, structured in seven phases: pre-engagement interactions, intelligence gathering, threat modelling, vulnerability analysis, exploitation, post-exploitation, and reporting. PTES provides the overarching procedural framework for this project: Lab 1 (reconnaissance and enumeration) maps to the intelligence gathering and vulnerability analysis phases, while Labs 2–4 progressively cover exploitation and post-exploitation techniques.

2.3.2 OWASP Top 10

The OWASP Top 10 is a regularly updated list of the most critical web application security vulnerabilities [11]. For educational purposes, it provides a focused, industry-recognised curriculum that prioritises the attack vectors students are most likely to encounter in practice. Lab 2 of this project is directly aligned with OWASP Top 10 categories.

2.3.3 CIS Critical Controls

The CIS Critical Controls (v8.0) represent a prioritised set of defensive techniques spanning network defence, data protection, and incident response [12]. Labs 1 and 3 incorporate defensive controls so that students understand not only how to attack but why specific security measures are implemented.

2.3.4 NIST Cybersecurity Framework

The NIST Cybersecurity Framework organises security activities into five functions: Identify, Protect, Detect, Respond, and Recover [13]. The lab design follows these principles by enabling students to identify vulnerabilities through hands-on exercises and understand corresponding protective measures.

2.4 Prior Academic Research

Academic research consistently validates the effectiveness of hands-on, laboratory-based approaches in cybersecurity education. Conti and Di Pietro [14] show that challenge-based environments significantly improve skill retention and practical competency compared to purely theoretical instruction. Murphy et al. [15] provide a structured methodology for designing laboratory exercises explicitly aligned with curriculum outcomes, validating the need for scenario-specific tasks and integrated assessment. Sponj and Heričko [16] conducted a systematic review of cybersecurity education research and identified hands-on practice and authentic task design as the two most influential factors in learner engagement and competency development. Al Mohanadi and Furnell [17] establish a requirements-driven approach to information security assessment laboratories, emphasising that lab tasks must map directly to defined learning outcomes to achieve meaningful pedagogical impact.

These findings directly inform the design choices in this project: each lab is built around scenario-based flags mapped to specific learning objectives, includes an teacher reference sheet, and culminates in an automated assessment mechanism. Together, these elements address the gap between platform availability and curriculum integration identified in the literature.

2.5 Gap Analysis: What Existing Solutions Miss

Despite the proliferation of cybersecurity training platforms, significant gaps remain:

Gap 1: Curriculum Integration. Most platforms target independent learners or certification preparation rather than integration into specific degree programmes. No major platform explicitly maps pentesting scenarios to the learning outcomes of a particular computer science curriculum.

Gap 2: Institution Control. Effective educational deployment requires institutional control over content, assessment, and infrastructure. External platforms introduce dependencies and limit customisation, while fully self-hosted solutions often impose high maintenance burdens.

Gap 3: Integrated Network Penetration Testing. Most platforms focus on isolated services or individual techniques. Few provide realistic, multi-system network topologies where reconnaissance informs a complete attack chain and lateral movement is required.

Gap 4: Automation and Scalability. Even classroom-oriented platforms often lack automated deployment, environment reset, and assessment mechanisms, which increases teacher workload and limits large-scale institutional adoption.

Gap 5: Balance of Guidance and Autonomy. Effective labs must balance sufficient structure to keep students on track with enough openness to encourage problem-solving and methodology development.

2.6 The Need for EVE-NG Based Labs

This analysis shows that while excellent individual components exist—gamified learning (TryHackMe), challenging scenarios (HackTheBox), research-backed content (SEED Labs), specialised domains (GOAD)—no existing solution comprehensively addresses the needs of a university cybersecurity education programme:

- **Curriculum-aligned labs**: Mapped to specific degree learning outcomes
- **Integrated network scenarios**: Multi-system penetration testing with realistic complexity

- **Institutional control:** Locally deployable, customisable, and maintainable
- **Complete automation:** One-command deployment and reset with built-in assessment
- **System-level realism:** Full virtual machine workflows (real OS images, ISO-based installation, native kernel behaviour) that reproduce the technical conditions of professional pentesting practice

This bachelor's thesis bridges that gap by using EVE-NG [1] to create a comprehensive, reusable, and automatable teaching package for cybersecurity practical labs. It draws on the pedagogical strengths of existing platforms while addressing their limitations through institutional control, curriculum alignment, and automation.

3 Objectives and Scope

This project aims to design, implement, and validate a reusable teaching package of practical cybersecurity labs based on EVE-NG[1]. The package enables students in the "Introduction to Cybersecurity" course to build practical skills in ethical pentesting through realistic and automated scenarios.

3.1 Specific Objectives (Measurable with KPIs)

3.1.1 OE1: Design and Implement EVE-NG Labs

Description: Create four functional and interconnected thematic labs.

Measurable KPIs:

- 4 fully functional .unl topologies (100%)
- Deployment time $\leq$ 2 minutes per lab (95% of cases)
- VM boot success rate $\geq$ 95%
- Compatibility with EVE-NG Community and Professional editions

3.1.2 OE2: Develop Automation Scripts

Description: Develop utility scripts that automate recurring lab management tasks, reducing manual effort for teachers and ensuring repeatable configurations across lab sessions.

Measurable KPIs:

- At least one functional automation script per lab covering a recurring task
- Scripts documented and reusable across different EVE-NG installations

- All scripts tested and verified in the project's EVE-NG environment

3.1.3 OE3: Develop Structured Teaching Material

Description: Develop comprehensive documentation for students and teachers.

Measurable KPIs:

- 4 student lab guides (1 per lab) in PDF format — task description, hints, and tool reference
- 4 teacher resolution guides (1 per lab) in PDF format — step-by-step walkthrough, expected output, and assessment rubric
- Assessment rubric included in each resolution guide with quantifiable criteria
- Lab setup procedure documented for teachers in each resolution guide

3.1.4 OE4: Validate Usability and Educational Effectiveness

Description: Verify that the package meets educational and technical requirements through a small pilot group representative of the target student population.

Measurable KPIs:

- Average lab resolution time within the 90–120 minute target window
- All pilot participants able to complete the lab with the provided guide
- User satisfaction score $\geq$ 4/5 (post-session survey)
- Zero blocking technical incidents during pilot sessions

Note: The pilot group comprises 4–5 individuals from the GEISI student population, potentially including participants with documented learning difficulties (e.g. dyslexia), to assess the effectiveness of the accessibility measures incorporated into the lab documentation.

3.2 Definition of the Client and Final User

3.2.1 Principal Client

- **Responsible professor:** Pere Vidiella i Catalan
- **Course:** Introduction to Cybersecurity (GEISI, Tecnocampus)
- **Context:** Undergraduate teaching in practical cybersecurity

3.2.2 Primary End Users

- **GEISI Students:** 25–30 students per edition
- **Profile:** Basic knowledge of networks and operating systems
- **Objective:** Learn ethical pentesting in a practical and safe manner

3.3 Scope Boundaries

Included in this bachelor's thesis:

- Design and implementation of 4 pentesting labs in EVE-NG
- Integration of real vulnerabilities and techniques aligned with OWASP Top 10[11] and CIS Critical Controls[12]
- Utility automation scripts for recurring lab management tasks (e.g. bridge configuration, ISO upload, connectivity checks)
- Complete technical documentation for teachers and future maintainers
- Student guides and learning objectives for each lab

- Assessment rubrics for evaluating student lab completion

Explicitly NOT included (out of scope):

- Integration with third-party Learning Management Systems (LMS) or Moodle
- Development of proprietary lab management software or web interfaces
- Creation of automated grading systems beyond basic script validation
- Commercial licensing or distribution models
- Formal pedagogical evaluation studies (e.g., pre/post learning assessments)
- Creation of labs beyond the four specified domains

3.4 KPIs and Performance Indicators

Category	Metric	Objective	Measurement Method
Technical	Lab deployment time	$\leq$ 2 min	Automated timing
Technical	VM boot success rate	$\geq$ 95%	System logs + validation scripts
Technical	Full reset time	$\leq$ 30 s	Scripts with timestamps
Technical	Scenario reproducibility	100%	Automated tests
Educational	Lab resolution time	90–120 min	Time tracking during pilot
Educational	Lab completion (pilot)	All participants	Observation during pilot sessions
Educational	User satisfaction	$\geq$ 4/5	Post-use survey
Educational	Key concept understanding	$\geq$ 80% correct	Assessment questionnaire
Quality	Documentation coverage	100% of features	Verification checklist
Quality	Critical errors	0	Exhaustive testing
Quality	EVE-NG compatibility	100%	Tests in multiple environments
Quality	Interface usability	$\geq$ 4/5	UX heuristic evaluation

Table 3.1: Key performance indicators for the lab teaching package

Note: KPIs are being measured during the pilot validation phase (April–May 2026) and will be documented in the final report.

Full deliverable specifications per objective, per-category KPI breakdowns, and extended audience analysis are provided in the project online documentation.

4 Methodology

This chapter outlines the research strategy, development approach, validation methods, and project lifecycle management used in this bachelor's thesis. The project combines principles from software engineering, cybersecurity pedagogy, and empirical research to ensure systematic development and validation of the EVE-NG[1]-based teaching package.

4.1 Research and Development Phases

The project is structured in four distinct phases:

4.1.1 Phase 0: Preparation and Infrastructure (July–November 2025)

Objective: Establish the technical foundation and personal capacity for the project. Activities included HomeLab infrastructure setup (Proxmox[18]), development environment configuration (VSCode[19], LaTeX[20], Git), and security tools installation (Parrot OS[21], Metasploit[22], Burp Suite[23]).

Duration: 105 hours

4.1.2 Phase 1: Preliminary Project Proposal and Conceptual Design (October 2025–January 2026)

Objective: Define project scope, objectives, and detailed planning. Work covered SMART objectives definition, state-of-the-art analysis of 7 platforms, functional and non-functional requirements specification, pentesting methodology selection (PTES[10], OWASP[24], CEH[25]), risk analysis with 12+ identified risks, resource allocation, and Gantt chart construction with 37 tasks across the critical path.

Duration: 188 hours **Milestone:** January 16, 2026

4.1.3 Phase 2: Implementation and Interim Validation (January–March 2026)

Objective: Develop the first three laboratories and conduct preliminary validation. Labs 1–3 were built and tested (reconnaissance, web vulnerabilities, and network/incident response), covering research, topology design, vulnerability implementation, and teaching material production. Informal pilot sessions with 4–5 GEISI students were run as each lab was completed, with usability observations, post-session surveys, and feedback-driven adjustments.

Duration: 212 hours **Milestone:** March 20, 2026

4.1.4 Phase 3: Completion and Final Validation (March–May 2026)

Objective: Develop Lab 4, complete automation scripts, and conduct final validation. Activities include privilege escalation lab development (CEH framework), utility script development (Bash/Python), formal pilot validation with the student group, and full bachelor's thesis report writing and revision.

Duration: 315 hours **Final Deadline:** May 27, 2026

4.2 Information Search Strategies

Research draws on academic databases (IEEE Xplore, ACM Digital Library, Scopus, Google Scholar) and primary industry standards: PTES[10], OWASP[24], NIST[13], [26], CIS Controls[12], and EC-Council CEH[25]. Platform and tool documentation was consulted for EVE-NG[27], Parrot OS[21], Metasploit[22], OWASP ZAP[28], and Burp Suite[23]. Primary research uses informal pilot sessions with GEISI students, post-session surveys, and quantitative metrics (time-on-task, completion rates).

4.3 Development Methodology

The project adopts an iterative, agile-inspired approach:

- **Design:** Conceptual architecture and topology planning
- **Implementation:** Lab creation, vulnerability implementation, tooling
- **Validation:** Security testing, educational effectiveness review
- **Iteration:** Feedback incorporation and refinement
- **Documentation:** Technical and pedagogical material preparation

All artefacts are version-controlled in a GitHub repository. Formal documentation is produced in LaTeX (this report, Volume I) and published as a MkDocs website for operational reference. Teacher deployment references, startup checklists, and credential tables are compiled separately in the Appendices (Volume II).

4.4 Platform Selection Rationale

A detailed comparison of existing cybersecurity training platforms is presented in Chapter 2 (State of the Art). That analysis concludes that EVE-NG is the most suitable choice for this project due to its open-source licensing, full topology customisation, local deployability, and automation support — advantages that directly address the gaps found in alternative platforms.

4.5 Validation and Success Criteria

Success is measured against the KPIs defined in Chapter 3, covering three dimensions: technical performance (deployment time, boot success rate, reproducibility), educational effectiveness (lab resolution time, completion rate, user satisfaction), and sustainability (documentation coverage, open-source compliance, institutional readiness).

Validation is conducted through pilot sessions with a representative group of GEISI students and through automated technical checks at each development milestone.

4.6 Risk Management

Risk	Mitigation Strategy	Owner
Scope creep	Clear scope boundaries and change management	Eloi
EVE-NG compatibility issues	Extensive testing on Community and Pro editions	Eloi
Performance degradation	Early benchmarking and optimisation iterations	Eloi
Documentation gaps	Peer review and user feedback cycles	Eloi + Faculty
Institutional adoption barriers	Change management plan and training support	Pere

Table 4.1: Key project risks and mitigation strategies

Full phase activity lists, quality assurance procedures, and documentation practices are provided in the project online documentation.

5 Functional and Non-Functional Requirements

This chapter specifies the requirements for the EVE-NG-based [1] cybersecurity teaching package. Functional requirements (RF) define what the system must do, technological requirements (TR) specify the technical environment, and non-functional requirements (NFR) address quality attributes and compliance.

5.1 Functional Requirements

5.1.1 RF1: Design and Implement EVE-NG Topologies

Description: Create four separate, functional, and interconnected network topologies in EVE-NG format (.unl files) that simulate real-world cybersecurity scenarios.

- **RF1.1:** Four topologies, each targeting a specific pentesting phase:
  - Lab 1: Reconnaissance (passive intelligence gathering)
  - Lab 2: Web application security (active vulnerability discovery)
  - Lab 3: Incident response and network forensics (lateral movement and log analysis)
  - Lab 4: Privilege escalation (post-exploitation scenarios)

Success Criteria: See technical KPIs in Chapter 3 (Table 3.1).

5.1.2 RF2: Implement Vulnerabilities and Attack Scenarios

Description: Implement realistic vulnerabilities aligned with PTES[10], OWASP[24], NIST[13], and CEH[25].

- **RF2.1**: Lab 1 — Reconnaissance: passive tools (Shodan, DNS enumeration), active scanning (Nmap[29], banner grabbing), service discovery and reporting.
- **RF2.2**: Lab 2 — Web Vulnerabilities (OWASP Top 10): SQL injection, command injection, authentication bypass, session flaws, XSS, CSRF, broken access control, and security misconfigurations.
- **RF2.3**: Lab 3 — Incident Response: lateral movement analysis, log forensics (auth.log, bash_history), privilege escalation via sudo misconfiguration, network traffic analysis with Wireshark[30].
- **RF2.4**: Lab 4 — Ransomware Response *(planned)*: students take the role of a system administrator responding to an active ransomware incident. Two of three servers remain accessible; the third is locked by encryption. Tasks cover identifying the attack vector, recovering encrypted files using different decryption techniques, and restoring access. Flags are obtained by successfully decrypting files or recovering credentials.

Success Criteria: Minimum 3–4 exploitable scenarios per lab, all resolvable with documented remediation procedures.

5.1.3 RF3: Create Structured Assessment Materials

Description: Develop assessment rubrics and evaluation materials aligned with the GEISI curriculum [31].

- **RF3.1**: 3–4 specific learning outcomes per lab, mapped to Bloom's taxonomy (knowledge, application, analysis) and GEISI degree competencies.
- **RF3.2**: Detailed assessment rubric per lab (100 pt scale with explicit criteria), included in the teacher resolution guide.

Success Criteria: 4 student lab guides and 4 teacher resolution guides (PDF), each including a rubric.

5.2 Technological Requirements

5.2.1 TR1: Virtualisation Infrastructure

- **TR1.1**: EVE-NG 5.0+ (Community and Professional editions), deployed as a VM on Proxmox VE; web interface via HTTPS; RESTful API for automation.

- **TR1.2**: Host specifications: 2× Intel Xeon E5-2680v4 (56 logical cores), 64 GB RAM, 2 TB NVMe storage, Gigabit Ethernet.

5.2.2 TR2: Security Tools and Penetration Testing Framework

- **TR2.1**: Attacking platform: Parrot OS Security 6.x [21] (primary). Selected over Kali Linux for its lower memory and disk footprint in concurrent EVE-NG deployments, cleaner QEMU integration, and a more approachable environment for students new to security distributions.

- **TR2.2**: Core tools on Parrot OS: Nmap, Metasploit 6.x[22], Burp Suite[23], OWASP ZAP[28], Wireshark[30], tcpdump, Python 3.10+, Bash.

- **TR2.3**: Target images: Ubuntu Server 24.04 LTS, Ubuntu Desktop 24.04 LTS, pfSense CE, VyOS, and the custom SYNAPSE web application (Lab 2).

5.2.3 TR3: Scripting and Automation

- **TR3.1**: Bash and Python 3.10+ for deployment, reset, and validation scripts.

- **TR3.2**: At least one utility script per lab; ISO uploader (`iso_uploader.py`), bridge configuration, and connectivity check scripts; all documented with usage instructions in the repository.

5.3 Non-Functional Requirements

NFR	Attribute	Key Requirements
NFR1	Performance	Deploy $\leq$2 min; VM boot $\leq$90 s (95%); reset $\leq$30 s; VM image $\leq$5 GB; $\leq$6 GB RAM per lab instance
NFR2	Compliance	NIST CSF functions (Identify/Protect/Detect/Respond/Recover); CIS Controls v8; OWASP Top 10 full coverage; PTES + CEH alignment
NFR3	Reliability	MTBF $\geq$100 h; MTTR $\leq$5 min; 100% scenario reproducibility; zero critical bugs at release
NFR4	Security	Full network isolation from production; no outbound internet from labs; RBAC for lab management; mandatory ethics agreement
NFR5	Usability	Faculty deploy time 5–10 min; student prerequisites minimal; WCAG 2.1 Level AA for web interfaces
NFR6	Sustainability	All components open-source or freely available; no vendor lock-in; explicit mapping to GEISI learning outcomes; horizontally scalable

Table 5.1: Non-functional requirements summary

5.4 Requirements Traceability Matrix

ID	Type	Requirement	Scope
RF1.1	Functional	Design 4 EVE-NG topologies	All labs
RF2.1	Functional	Lab 1 reconnaissance scenarios	Lab 1
RF2.2	Functional	Lab 2 OWASP vulnerabilities	Lab 2
RF2.3	Functional	Lab 3 incident response / forensics	Lab 3
RF2.4	Functional	Lab 4 privilege escalation	Lab 4
RF3.1	Functional	Learning outcomes defined	All labs
RF3.2	Functional	Assessment rubrics	All labs
TR1.1	Technical	EVE-NG 5.0+	Platform
TR1.2	Technical	Host 64 GB RAM, 2 TB storage	Infrastructure
TR2.1	Technical	Parrot OS Security as attacking platform	Tools
NFR1	Non-Func.	Deploy $\leq$2 min, reset $\leq$30 s	Performance
NFR2	Non-Func.	NIST / CIS / OWASP / PTES alignment	Compliance
NFR4	Non-Func.	Network isolation + ethics	Security
NFR6	Non-Func.	Open-source, GEISI aligned	Sustainability

Table 5.2: Requirements traceability matrix

Detailed sub-requirements for RF1.2–RF1.4, RF2.5, RF3.3–RF3.4, TR1.3, TR1.4, TR2.4, TR3.3, and the full NFR subsections are provided in the

project online documentation.

6 Feasibility Study

6.1 Gantt Chart and Task Definition

The project comprises **37 tasks** across 7 categories, totalling 820 hours (20 ECTS × 25 h/ECTS). The detailed phase breakdown and activity descriptions are provided in the Methodology chapter (Chapter 4).

Category	Tasks	Hours
Preparation	4	105h
Analysis	5	49h
Documentation	6	95h
Development	8	178h
Testing	5	123h
Implementation	6	175h
Milestones	3	95h
TOTAL	**37**	**820h**

Table 6.1: Distribution of tasks by category

The complete Gantt chart was built with a custom interactive web tool available at `docs/resources/gantt/index.html` in the project repository.

6.1.1 Critical Path

The project has **13 critical tasks** (277 h total) that directly affect milestone deadlines:

ID	Critical Task	Hours	Deadline
1.4	Pentesting Methodology and Validation	8h	Nov 26, 2025
1.10	Milestone: Preliminary Project	40h	Jan 16, 2026
2.1	Lab 1 Research	14h	Jan 26, 2026
2.2	Lab 2 Research	14h	Jan 31, 2026
2.3	Lab 3 Research – Network Analysis & IR	14h	Feb 7, 2026
2.6	Pre-validation with Pilot Users	20h	Feb 23, 2026
2.6.1	Pre-validation Revision	6h	Mar 1, 2026
3.1	Lab 4 Research	16h	Mar 23, 2026
3.4	Penetration Testing Round 1	30h	Apr 16, 2026
3.5	Penetration Testing Round 2	25h	Apr 30, 2026
3.9	Bachelor's thesis Report Writing I	15h	May 1, 2026
3.10	Bachelor's thesis Report Writing II	40h	May 20, 2026
3.11	Bachelor's thesis Report Writing III	35h	May 25, 2026
TOTAL CRITICAL PATH		**277h**	

Table 6.2: Critical path tasks and deadlines

6.2 Budget

6.2.1 Human Resources

Phase	Hours	Rate	Total
Phase 0: Preparation	105h	15€/h	1,575€
Phase 1: Preliminary Project	188h	15€/h	2,820€
Phase 2: Interim	212h	15€/h	3,180€
Phase 3: Final	315h	15€/h	4,725€
TOTAL	**820h**		**12,300€**

Table 6.3: Human resources cost (junior engineer rate, 15€/h, theoretical)

6.2.2 Hardware and Software

Resource	Qty	Unit	Total
HomeLab Server (2× Xeon E5-2680v4, 64 GB, HDD+SSD)	1	800 €	800 €
Development Workstation	1	1,200 €	1,200 €
Network Equipment	1 set	150 €	150 €
External Storage	1	100 €	100 €
SUBTOTAL Hardware			**2,250 €**

Table 6.4: Hardware costs

Resource	License	Cost
EVE-NG Community Edition	Free/Open Source	0 €
Parrot OS Security	Free/Open Source	0 €
Metasploit Framework	Free/Open Source	0 €
Burp Suite Community	Free	0 €
OWASP ZAP	Free/Open Source	0 €
Wireshark	Free/Open Source	0 €
Visual Studio Code	Free	0 €
LaTeX (TeX Live)	Free/Open Source	0 €
Git + GitHub	Free (Student)	0 €
SUBTOTAL Software		**0 €**

Table 6.5: Software and license costs (all free/open-source for educational use)

The server is a one-time investment reused across multiple projects. An additional 32 GB RAM module (~80 €) was acquired during the project and is not reflected in the original estimate.

6.2.3 Other Costs

Concept	Cost
Electricity consumption (820 h × 0.15 kWh × 0.20 €/kWh)	25 €
Internet connectivity (9 months × 40 €/month)	360 €
Documentation printing and binding (3 copies)	60 €
Contingency fund (5% of total)	635 €
TOTAL Other Costs	**1,080 €**

Table 6.6: Additional operational costs

6.2.4 Total Budget

Category	Total
Human Resources	12,300 €
Hardware	2,250 €
Software	0 €
Other (electricity, internet, contingency)	1,080 €
TOTAL	**15,630 €**

Table 6.7: Complete project budget

6.3 Feasibility Analysis

6.3.1 Technical Feasibility

Phase 0 infrastructure is 100% operational (EVE-NG, Proxmox[18], Parrot OS[21], development environment). All software is open-source and widely adopted in professional cybersecurity training. The required competencies are:

Area	Required Competency	Level
Network Administration	VLANs, routing, firewall rules	Intermediate
Operating Systems	GNU/Linux administration (Debian/Ubuntu)	Intermediate
Virtualisation	EVE-NG topology design, VM management	Advanced
Pentesting	Reconnaissance, exploitation, web vulns	Intermediate
Scripting	Python and Bash for automation	Intermediate
Documentation	LaTeX, technical writing	Intermediate

Table 6.8: Required technical competencies

Risk assessment follows NIST SP 800-30[32]. Table 6.9 summarises the six identified risks with probability, impact, and mitigation strategy.

Risk	Prob.	Impact	Level	Mitigation
EVE-NG Performance Issues	Med	High	HIGH	Dedicated 32 GB RAM; VirtualBox fallback
VM Image Incompatibility	Med	Med	MEDIUM	Early validation; QCOW2 standardisation
Knowledge Gaps (advanced net.)	Med	Med	MEDIUM	Structured learning; tutor consultation
Hardware Degradation	Low	High	MEDIUM	Quarterly diagnostics; S.M.A.R.T. alerts
Data Loss (backup insufficient)	Low	High	MEDIUM	Daily backups; Git + off-site cloud
Configuration Gaps	Med	Med	MEDIUM	Continuous docs; topology exports via Git

Table 6.9: Risk assessment matrix (NIST SP 800-30). No risk is project-blocking.

Conclusion: High technical feasibility. All risks have concrete mitigations and none is project-blocking.

6.3.2 Economic Feasibility

Alternative	Description	Annual Cost
This Project	Custom EVE-NG labs, curriculum-aligned	**0 €/year***
HackTheBox Academy	Commercial labs platform	1,200 €/year
TryHackMe Education	Gamified learning paths	800 €/year
Cybrary for Teams	Video training + labs	1,500 €/year
Custom Enterprise Labs	Outsourced development	8,000–15,000 €

Table 6.10: Cost comparison with commercial alternatives (*excludes initial development)

Conclusion: High economic feasibility. Zero recurring software costs; hardware amortised over 3–5 years; comparable commercial alternatives cost 800–1,500 €/year.

6.3.3 Temporal Feasibility

The project spanned 10 months (July 2025–May 2026) across four phases totalling 820 hours (20 ECTS $\times$ 25 h/ECTS), as shown in Ta-

ble 6.1. All milestones were met on schedule: Phase 0 (infrastructure), Phase 1 (preliminary proposal), Phase 2 (Labs 1–3 implementation and interim validation), and Phase 3 (Lab 4, final validation, and report writing). The 820-hour workload was distributed as planned, with the critical path of 13 tasks (277 h, Table 6.2) completed without blocking delays.

Conclusion: High temporal feasibility. The project was delivered within the planned timeline and within the 820 h budget for 20 ECTS.

6.3.4 Environmental Feasibility

The project's energy footprint comes mainly from the HomeLab server (150 W avg.) and workstation (65 W avg.). Development phase consumption: 188.6 kWh; operational phase (4 sessions × 30 students × 3 h/year): 82.8 kWh/year. A 5 kWp rooftop solar installation at the author's home generated 6,634.9 kWh in 2025 (real data), offsetting approximately 34% of development-phase consumption and reducing CO_2 emissions by 32% compared to grid-only scenarios[33], [34].

Scenario	Description	CO_2/year	With Solar
This Project (ops)	Virtualised shared HomeLab	20.7 kg	14.1 kg
This Project (dev)	Development phase, solar-offset	47.2 kg	32.0 kg
Individual VMs	30 students × personal laptop	180 kg	–
Cloud-Based Labs	AWS/Azure (PUE 1.5)	95 kg	–
Physical Lab	Dedicated hardware switches/routers	450 kg	–

Table 6.11: CO^2 comparison across laboratory approaches

Sustainability measures: VMs suspended when idle; digital-only documentation (3 printed copies required by regulation); modular server hardware designed for 3–5 year longevity; open-source stack extends the usable lifespan of existing hardware.

Conclusion: Good environmental feasibility; significantly lower carbon footprint than physical labs or individual student deployments.

6.3.5 Legal Aspects

Intellectual Property and Licensing

All software (EVE-NG, Parrot OS Security, Metasploitable, DVWA) is licensed under permissive open-source licenses (GPL, MIT, Apache 2.0) that permit educational use, modification, and redistribution. Per UPF regulations (Article 10), intellectual property of the bachelor's thesis belongs to the student; exploitation rights default to the student unless otherwise agreed with the tutor and institution. All citations follow IEEE standards.

Ethical and Legal Considerations for Pentesting

All pentesting activities are conducted within isolated EVE-NG sandboxes with no connection to production networks. Only intentionally vulnerable systems (Metasploitable, DVWA[35]) designed for training are used as targets. The project is conducted under academic supervision (Pere Vidiella i Catalan) within the scope of the "Introduction to Cybersecurity" curriculum. No personally identifiable information is processed; all scenarios use synthetic data.

Use of Artificial Intelligence Tools

Auxiliary AI tools — both locally deployed language models on the HomeLab and complementary online services — were used only to review draft text, detect inconsistencies, and check documentation errors. No AI tool was used to generate original technical content, conclusions, research findings, or design decisions; all creative, analytical, and engineering work is the author's own. This disclosure is provided in accordance with the academic integrity guidelines applicable to the programme.

Compliance with Academic Regulations

The project follows Tecnocampus ESUPT bachelor's thesis regulations (approved September 30, 2021): original work with IEEE citation standards (Article 11 on plagiarism); structure aligned with Chapter 9 evaluation requirements (continuous assessment: preliminary project proposal, interim, final delivery, tribunal defence); formatting per the ESUPT redaction guide.

6.3.6 Gender and Diversity Perspective

Inclusive Language and Representation

All documentation — thesis, technical write-ups, and student guides — uses gender-neutral language. Fictional accounts, scenario names, and user credentials in lab environments are culturally neutral and carry no gender or demographic assumptions. Lab scenarios use diverse names that reflect the multicultural nature of the cybersecurity profession.

Addressing the Gender Gap in Cybersecurity

Cybersecurity remains a male-dominated field, with women representing around 25% of the global workforce[36]. This project contributes to lowering barriers to entry in two ways. First, well-structured and clearly documented labs reduce the intimidation factor often associated with hands-on security tools, making the material more approachable for students from diverse backgrounds. Second, the ethical framing of the course — emphasising responsible disclosure, defensive security, and societal impact — broadens the appeal of cybersecurity beyond a narrow stereotype, which research suggests helps attract a more diverse student population[37].

Accessibility Measures

The practical barriers to engagement in this course are not demographic but relate to two specific factors:

- **Prior knowledge:** Students with limited background in networking or operating systems may find early labs more demanding. This is mitigated through structured guides with progressive hints and a tools reference section in each student PDF.
- **Reading difficulties (dyslexia):** Dense technical documentation can be harder to process for dyslexic readers. This thesis is typeset in OpenDyslexic; lab documentation uses short paragraphs, descriptive headings, and visually separated code blocks. The pilot study confirmed that a participant with documented dyslexia was able to complete Lab 3 with the materials provided.

Conclusion: Inclusion measures are grounded in the real barriers identified for this academic context — knowledge scaffolding and typographic accessibility — while also addressing the broader gender gap through ethical framing and accessible entry points.

Overall Feasibility. The project is feasible across all evaluated dimensions. Technical risks have been mitigated through hardware investment (64 GB RAM, additional module acquired during development). The budget is within academic scope and the cost model is transparent. Legal and ethical obligations are satisfied through open-source licensing and a locally controlled infrastructure. All planned activities fit within the TecnoCampus normative timeline. No identified risk is considered project-blocking.

Full phase task lists (Task IDs, hours, critical flags), the five-phase NIST SP 800-30 risk assessment, and detailed budget breakdowns are provided in the project online documentation.

7 Development and Implementation

This chapter describes the implementation of the penetration testing lab environment built on EVE-NG. It covers the platform setup, virtual machine image management, the lab management interface, infrastructure problems encountered during development, and an overview of the lab topologies designed for the course. Each lab is described in its own section, with a summary of the network topology, techniques, and problems encountered. Detailed teacher references — including deployment credentials, startup procedures, and flag locations — are compiled in Volume II (Appendices B–D) and the project online documentation. The EVE-NG installation procedure is documented in the project online documentation.

7.1 Platform Setup

The development process began with the installation and validation of EVE-NG [1] as the network emulation platform. Following the official EVE-NG documentation, an initial environment was deployed to verify basic functionality: creating simple network topologies, deploying virtual machines with different operating systems, and testing console and remote desktop access before committing to the full lab design.

Several attempts were made to run EVE-NG as a virtual machine on a VMware-based infrastructure available in the university environment. Despite VMware being an officially supported hypervisor, recurring stability and integration issues prevented a reliable setup. After evaluating the available alternatives, and in agreement with the project supervisor, the platform was migrated to a dedicated Proxmox VE [18] server. This choice offers better compatibility with the nested virtualisation required by EVE-NG and can be replicated on any institutional server running the same hypervisor.

On top of Proxmox, an EVE-NG virtual machine was deployed following the official installation guidelines, with the networking model

adapted to the Proxmox bridge configuration. Console access via Telnet and graphical access via VNC were validated successfully for all node types used in the labs. Native Wireshark integration was not enabled in this iteration due to conflicts with pre-existing software on the host machine, and was therefore left out of scope for the current version.

7.2 Image Management

7.2.1 Image Selection

Once the platform was stable, a survey of available EVE-NG-compatible images was conducted, prioritising free and open-source operating systems suitable for an academic context. Starting from a broad list of candidates, the final selection was reduced to a minimal, reusable core for the penetration testing scenarios:

- **VyOS** [38]: used as a router and Layer 3 switch for routing and inter-VLAN scenarios.
- **pfSense CE** [39]: used as the perimeter firewall, providing NAT, DNS resolution, and port forwarding.
- **GNU/Linux distributions** (Ubuntu Server 24.04 [40], Ubuntu Desktop 24.04, Parrot OS [21]): used as target servers, internal workstations, and attacker machines respectively.

This reduced set balances realism and manageability, keeps all software licences aligned with the academic context, and limits the overhead of maintaining multiple image versions across labs.

Parrot OS Security Edition was selected over the more widely known Kali Linux [41] after evaluating both options during image integration. The Kali image proved substantially heavier, required additional QEMU guest configuration to work reliably inside EVE-NG, and produced higher per-instance resource overhead. Parrot OS provides an equivalent toolset — Nmap, Metasploit, Burp Suite, Hydra, Gobuster, and all tools required by the labs — with a lighter footprint and

cleaner EVE-NG integration. An additional consideration was accessibility: Parrot OS includes OpenDyslexic and other accessibility fonts by default, consistent with the inclusive design principles adopted for the lab documentation. This decision is reflected in the requirements specification (Section TR2.1).

7.2.2 Image Organisation in EVE-NG

To ensure consistency and reproducibility, all images were organised under the standard EVE-NG path `/opt/unetlab/addons/qemu/`. Each system was placed in a dedicated directory with a normalised name following the pattern `<type>-<version>` (e.g., `pfsense-2.7.2`, `vyos-1.x-rolling-amd64`, `linux-ubuntu-server-24.04`).

For GNU/Linux systems, the `linux-` prefix was applied consistently (e.g., `linux-ubuntu-desktop-24.04`, `linux-parrot-rolling`), since EVE-NG only makes the *Linux* node category available in the interface when at least one directory with this prefix exists. Each directory contains a bootable image named `cdrom.iso` and a `virtioa.qcow2` disk image sized according to the role of the node.

7.2.3 ISO Upload Automation

The manual workflow for preparing each image — creating the directory, uploading the ISO, renaming it, creating the disk, and fixing permissions — proved repetitive and error-prone during initial experiments. To address this, a Python script named `iso_uploader.py` was developed as part of the project tooling.

The script presents a graphical file picker, uploads the selected ISO files to the EVE-NG server via SCP, and then automates the remaining preparation steps: directory creation under `/opt/unetlab/addons/qemu/`, renaming the ISO to `cdrom.iso`, creation of the `virtioa.qcow2` disk at a size inferred from the image type, and execution of `unl_wrapper -a fixpermissions`. This tool reduces the time required to onboard a new image and enforces a uni-

form directory structure across deployments.

7.3 EVE-NG Lab Management Interface

EVE-NG provides a web-based graphical interface through which administrators and students can manage lab files independently. The *File Manager*, accessible after login, is the main entry point for lab operations and offers several features relevant to reproducibility and multi-user deployments:

- **Lab import and export:** Topology files (`.unl`) can be imported by dragging them into the interface. Exported archives preserve the topology, node configurations, and snapshots, making it straightforward to share or back up a complete lab.
- **Folder structure:** Labs can be organised hierarchically (e.g., `/GEISI-Labs/Pentesting/`), which keeps different courses and iterations separate without requiring multiple EVE-NG installations.
- **Multi-lab support:** Several topologies can run simultaneously. The administrator dashboard provides a real-time view of CPU and memory consumption per lab, which is useful for capacity planning on shared infrastructure.

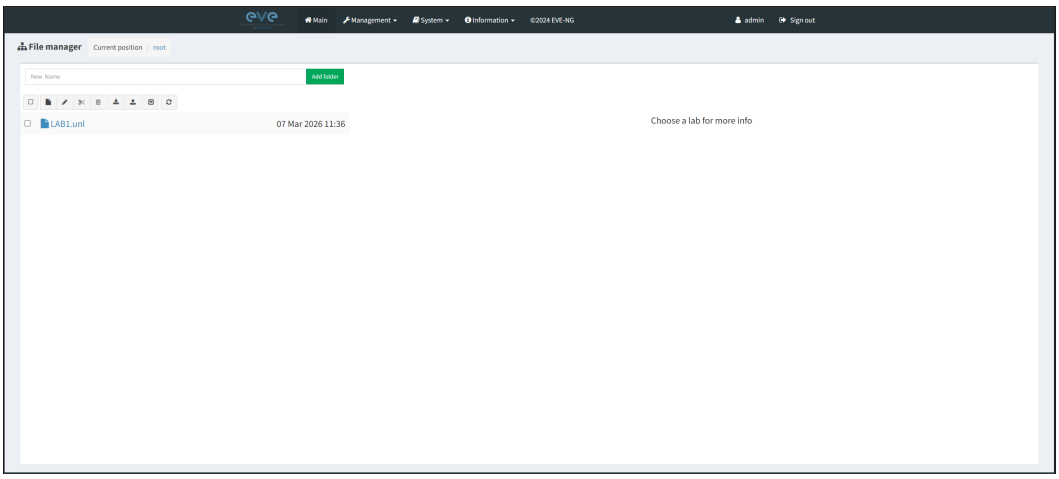

Figure 7.1: EVE-NG File Manager showing the lab directory structure, import controls, and topology thumbnails.

The export format also simplifies distribution: a `.unl` file committed to the project repository can be imported into any compatible EVE-NG instance, reproducing the full topology without manual reconfiguration.

7.4 Infrastructure Problems

Several problems were encountered at the infrastructure level during the setup of the EVE-NG environment. These are distinct from the lab-specific issues documented within each lab section and relate to the underlying platform and tooling.

7.4.1 EVE-NG on VMware

The initial deployment target was a VMware-based server available through the university. Despite EVE-NG being officially listed as compatible with VMware [42] hypervisors, the environment produced recurring instability: nodes failed to start consistently, VNC sessions dropped without warning, and network connectivity between virtual bridges was unreliable. No stable configuration was found after several iterations. The root cause was not fully identified, but the combination of VMware's nested virtualisation restrictions and the specific hardware and software versions in the university environment appeared to be contributing factors. Migrating to Proxmox resolved all of these issues.

7.4.2 Node Configuration Persistence

A recurring challenge in EVE-NG lab design is ensuring that node configurations survive restarts. By default, runtime network settings applied inside guest virtual machines are lost when the lab is stopped or the EVE-NG host is rebooted. This affected multiple nodes across the labs and required a dedicated solution for each one. The specific mechanisms used are described in the relevant lab sections; the general principle adopted throughout the project was to rely on the native persistence mechanisms of each operating system — such as `systemd-networkd` unit files, NetworkManager connection profiles,

and udev rules — rather than applying settings interactively at the console.

7.4.3 LVM Volume Group Naming Conflict

During a session involving disk operations on guest virtual machine images mounted from the EVE-NG host, a naming conflict arose between the host's LVM volume group and a guest disk's volume group, both named ubuntu-vg. Attempting to resolve the conflict by renaming the host volume group caused the EVE-NG host to fail on the next restart, dropping into an initramfs prompt because the boot configuration still referenced the original group name.

Recovery was performed from the initramfs shell by renaming the volume group back to its original name and reactivating it before resuming the normal boot sequence. As a result of this incident, the rule established for the project is to never rename a volume group belonging to the host operating system. When a naming conflict arises with a guest disk, only the guest volume group is renamed, using a descriptive identifier that includes the lab and node reference.

7.4.4 Memory Capacity Constraint

During concurrent development of the first two laboratories, peak memory consumption on the EVE-NG virtual machine regularly exceeded the initial 32 GB RAM allocation of the host server. Running pfSense, VyOS, Ubuntu Server, Ubuntu Desktop, and Parrot OS simultaneously — alongside other services co-hosted on the same Proxmox infrastructure (monitoring stack, reverse proxy, storage services) — produced memory pressure that caused nodes to be killed by the hypervisor's out-of-memory handler, interrupting development sessions.

An additional 32 GB RAM module was acquired and installed, bringing the host to 64 GB total. This resolved the stability issues and allowed all lab nodes to run concurrently without further intervention. The upgrade was not included in the original project budget estimate; its approximate cost (~80 €) is noted in Chapter 6 as a post-planning

expenditure.

7.5 Lab Topologies

Four penetration testing lab topologies were designed for the course. Each topology is implemented as an EVE-NG `.unl` file and can be imported into any compatible EVE-NG instance. Table 7.1 summarises the four labs, their primary learning objective, and their current implementation status.

Lab	Primary Objective	Status
Lab 1: Reconnaissance & Enumeration	Port scanning, web content discovery, SSH	Implemented
Lab 2: Web Application Vulnerabilities	OWASP Top 10 exploitation	Implemented
Lab 3: Network Analysis	Traffic capture, lateral movement, incident response	Implemented
Lab 4: Privilege Escalation	Local privilege escalation techniques	Pending

Table 7.1: Overview of the four EVE-NG lab topologies designed for the course.

The remainder of this chapter describes each implemented lab in detail, covering the network topology, design decisions, student workflow, and problems encountered during development.

7.6 Lab 1: Reconnaissance and Enumeration

Lab 1 introduces the reconnaissance phase of a penetration test following the PTES framework [10]. Students interact with a fictional target organisation, *PEBCAK Corp*, through a layered topology that includes a perimeter firewall (pfSense CE [39]), an internal router (VyOS [38]), and a target Ubuntu Server 24.04 [40]. Access is exclusively via DNS hostname (`pebcak.local`) resolved through pfSense's Unbound resolver, requiring students to use proper service enumeration before any exploitation attempt. Full topology diagrams, node configuration, student guide, and teacher walkthrough

are in the project online documentation; interactive documentation at `docs/web/docs/labs/lab1/`.

Network Addressing

Node	Role	Interface	IP	Network
Parrot-OS	Attacker	eth0	10.0.0.10	10.0.0.0/24 (WAN-sim)
pfSense	Firewall/NAT	WAN	10.0.0.1	10.0.0.0/24
pfSense	Firewall/NAT	LAN	192.168.1.1	192.168.1.0/24
VyOS	Router/L3	eth0	192.168.1.2	192.168.1.0/24
VyOS	Router/L3	eth1	10.10.10.1	10.10.10.0/24
Server-PC1	Target	eth0	10.10.10.10	10.10.10.0/24

Table 7.2: Lab 1 IP addressing scheme

Techniques and Framework Alignment

Technique	Description	Reference
DNS resolution	Hostname-to-IP via Unbound resolver (RFC 1035 [43])	PTES Recon
Port scanning	TCP SYN scan with Nmap (RFC 793 [44])	PTES Recon
Service enumeration	Banner grabbing, version detection	PTES Recon
Web content discovery	Directory brute-force with Gobuster	OWASP WSTG
Credential retrieval	Exposed credentials in web content	OWASP A05
SSH access	Password-based login (RFC 4252 [45])	PTES Exploit

Table 7.3: Lab 1 techniques and framework mapping

7.6.1 Problems Encountered and Solutions

VNC Keyboard Input on QEMU Nodes

EVE-NG provides VNC console access to QEMU nodes, but certain key combinations — most notably `Ctrl+C` and special characters such as |, @, and { — are not reliably forwarded by the browser-based VNC client or by the `vncdo` command-line tool. The root cause is

that vncdo sends key symbols rather than raw key codes, and the VNC server's key mapping does not cover all combinations used in a penetration testing context.

The solution was to inject keystrokes directly via the QEMU monitor socket, using Python's raw socket interface to send `sendkey` commands. This bypasses the VNC protocol entirely and ensures that arbitrary key sequences, including those involving modifier keys, are delivered correctly to the guest.

pfSense DNS Resolver WAN Interface

After initial configuration, DNS queries from the attacker machine were not being resolved by pfSense's Unbound resolver. The problem was that Unbound, by default, only listens on the LAN interface, so queries arriving on the WAN-side interface were silently discarded. The fix was to explicitly add the WAN interface to Unbound's listening configuration under *Services* → *DNS Resolver* → *Network Interfaces*.

Unbound Startup Configuration Path

On subsequent restarts, the Unbound configuration applied via the GUI was not being preserved, causing DNS resolution to fail after every reboot. Investigation revealed that pfSense's startup scripts write the Unbound configuration to `/var/unbound/unbound.conf` at boot time, overwriting any manual edits. The correct approach is to configure all resolver settings exclusively through the GUI or through pfSense's `/etc/inc/` PHP hooks, never by editing the generated configuration file directly.

VyOS Source NAT Requirement

Traffic routed through VyOS from the attacker to the target was being dropped because the target's default gateway was pfSense, not VyOS. Return packets were therefore routed back via pfSense rather than through VyOS, breaking the connection. The fix was to

configure Source NAT (masquerade) on VyOS for traffic entering the internal segment, ensuring that return traffic would follow the same path back through the VyOS router.

Results

Flag	Location	Technique
FLAG{recon_enum_success}	`/flag.txt` on Server-PC1 via SSH	Credential reuse
FLAG{pebcak_firewall_pivot}	pfSense admin panel (optional)	NAT pivot

Table 7.4: Lab 1 flags and acquisition techniques

7.7 Lab 2: Web Application Vulnerabilities — SYNAPSE Intelligence Portal

Lab 2 places students in front of a custom two-tier web application, *SYNAPSE Intelligence Portal*, deployed on a segmented DMZ architecture. The scenario is aligned with the OWASP Top 10 [11] and chains multiple vulnerabilities across two servers: Server-A hosts the portal front-end with authentication and reporting features; Server-B provides an internal monitoring API accessible only via Server-A. Students must complete a progressive exploitation chain — stored XSS, cookie theft, SQL injection, and command injection — to collect all three flags. Full topology, source code, credentials, and startup procedure are in the project online documentation; interactive documentation at `docs/web/docs/labs/lab2/`.

Network Addressing

Node	Role	Interface	IP	Network
Parrot-OS	Attacker	eth0	10.0.2.10	10.0.2.0/24 (WAN-sim)
pfSense-LAB2	Firewall/NAT/DNAT	WAN	10.0.2.1	10.0.2.0/24
pfSense-LAB2	Firewall/NAT	LAN	192.168.2.1	192.168.2.0/24
VyOS-LAB2	DMZ Router	eth0	192.168.2.2	192.168.2.0/24
VyOS-LAB2	DMZ Router	eth1	10.0.3.1	10.0.3.0/24 (DMZ)
Server-A	Web front-end	eth0	10.0.3.10	10.0.3.0/24
Server-B	Internal API	eth0	10.0.3.20	10.0.3.0/24

Table 7.5: Lab 2 IP addressing scheme

OWASP Coverage and Vulnerability Chain

Step	OWASP	Vulnerability	Flag
1	A03:2021	Stored XSS in report field → session cookie theft	–
2	A07:2021	Cookie injection → admin panel access	–
3	A03:2021	SQL injection in search → credential dump	FLAG{synapse_sqli_creds_dumped}
4	A01:2021	Lateral movement to Server-B via stolen creds	FLAG{synapse_pivot_server_b}
5	A03:2021	Command injection in monitoring API	FLAG{synapse_rce_achieved}

Table 7.6: Lab 2 OWASP Top 10 vulnerability chain

7.7.1 Problems Encountered and Solutions

pfSense Block Private Networks and Reply-To

Two pfSense default behaviours blocked traffic in the lab topology. First, the *Block private networks* option on the WAN interface was enabled by default, causing pfSense to silently discard packets arriving from the simulated attacker network (RFC 1918 space [46]). Second, the *Reply-to* feature added return routes in `pf` rules that conflicted

with VyOS's routing table, causing asymmetric routing. Both were disabled: the private network block via the WAN interface settings, and reply-to via a per-rule option in the firewall rule editor.

VNC Keyboard Input on pfSense

The same VNC key injection issue documented in Lab 1 reappeared on pfSense console sessions, particularly when entering characters required by the pfSense shell (|, &, $). The QEMU monitor raw socket approach was applied identically.

Server-A Volume Group Conflict

When mounting the Server-A QCOW2 image from the EVE-NG host to pre-stage application files, the host's own LVM volume group (`ubuntu-vg`) conflicted with the guest disk's identically named group. Activating both simultaneously caused the host's logical volumes to be shadowed. The solution was to rename the guest volume group to `synapse-vg` using `vgrename` before activating it on the host, then rename it back after dismounting.

Server-B Installer Keyboard Layout

During the Server-B installation, the Ubuntu installer defaulted to a Spanish keyboard layout due to the locale of the EVE-NG host, causing mismatched characters when entering the initial credentials. This was corrected by explicitly selecting the `en_US` keyboard layout at the installer prompt and verified with a `/dev/urandom`-seeded password test before proceeding.

Static Route Gateway Reference

VyOS rejected a static route configuration that referenced a gateway IP not directly reachable on any configured interface. The intended next-hop was on a subnet not yet assigned to any VyOS interface

at the time the route was applied. This was resolved by defining the interface address before the static route, and by using `commit` and `save` in the correct order within the VyOS configuration session.

7.8 Lab 3: Incident Response and Log Forensics — HELIX Systems

Lab 3 inverts the student's role: instead of attacking, they act as a security analyst responding to an active incident at *HELIX Systems*. The lab environment comes pre-staged with evidence of a simulated intrusion — brute-force SSH attempts, a successful login, privilege escalation via `sudo` misconfiguration, backdoor account creation, and lateral movement to a database server. Students must reconstruct the attack timeline from system logs (`auth.log`, `bash_history`) and identify the attacker's actions and impact, following a methodology aligned with NIST SP 800-61 [47]. Full evidence walkthrough, account table, and attack timeline are in the project online documentation; interactive documentation at `docs/web/docs/labs/lab3/`.

Network Addressing

Node	Role	Interface	IP	Network
Parrot-OS	Analyst station	eth0	10.0.4.10	10.0.4.0/24 (WAN-sim)
pfSense-LAB3	Firewall/DNAT	WAN	10.0.4.1	10.0.4.0/24
pfSense-LAB3	Firewall	LAN	192.168.3.1	192.168.3.0/24
VyOS-LAB3	Internal router	eth0	192.168.3.2	192.168.3.0/24
VyOS-LAB3	Internal router	eth1	10.0.5.1	10.0.5.0/24 (corp)
Server-Web	Compromised web	eth0	10.0.5.10	10.0.5.0/24
Server-DB	Internal DB	eth0	10.0.5.20	10.0.5.0/24

Table 7.7: Lab 3 IP addressing scheme

MITRE ATT&CK Coverage

MITRE ID	Technique	Evidence in lab
T1110.001	Brute Force: Password Guessing	`auth.log` failed attempts
T1078	Valid Accounts	`devops` account compromise
T1548.003	Sudo and Sudo Caching	`find` NOPASSWD in sudoers
T1136.001	Create Account: Local	Backdoor `sysbackup` account
T1021.004	Remote Services: SSH	Lateral movement to Server-DB
T1005	Data from Local System	`/opt/helix/data/` exfil marker

Table 7.8: Lab 3 MITRE ATT&CK technique coverage

7.8.1 Problems Encountered and Solutions

LVM Volume Group Naming Conflict

When pre-staging Server-Web's evidence files from the EVE-NG host, the same volume group conflict from Lab 2 recurred: both the host and the guest disk used `ubuntu-vg`. The resolution procedure was identical — renaming the guest VG to `helix-web-vg` before activation — and the naming rule was formally added to the project's infrastructure runbook to prevent recurrence in future labs.

Read-only nbd Device

On one occasion, the QCOW2 image mounted via `qemu-nbd` was presented as read-only despite the `-readwrite` flag. Investigation revealed that a previous mount session had not been cleanly disconnected (`qemu-nbd --disconnect`), leaving the kernel's NBD client in an inconsistent state. The fix was to explicitly disconnect any stale NBD device before re-mounting, and to add a pre-mount check to the project's image manipulation scripts.

analyst Group Membership for Log Access

The student account (`analyst`) was created with restricted permissions so that it could not read system logs directly as root. However, `/var/log/auth.log` requires membership of the `adm` group on Ubuntu. The pre-staging script was updated to add `analyst` to `adm` at image preparation time, preserving the educational constraint (no `sudo` to `root`) while allowing access to the evidence files that are the focus of the lab.

pfSense Console Access via QEMU Monitor

The pfSense serial console was inaccessible through the standard EVE-NG Telnet interface due to a configuration mismatch between the QEMU serial port definition and pfSense's expected console device (`ttyu0` vs `ttyS0`). Console access was recovered by connecting directly to the QEMU monitor socket and issuing a `sendkey` sequence to navigate the pfSense boot menu, where the console port assignment could be corrected persistently.

7.9 Pilot Validation

A small pilot study was carried out during April–May 2026 to test whether the labs work well in practice and whether students can complete them using the provided guides.

7.9.1 Pilot Design

Five participants took part in the pilot. All of them are currently studying cybersecurity, so they have some technical background, but they are not experts in all the topics covered by the labs. The author was available during the sessions to answer questions when participants got stuck, so the study reflects a guided but realistic classroom setting rather than fully independent self-study.

One participant has documented dyslexia. This made it possible to

get some early feedback on the accessibility features of the guides, such as the use of OpenDyslexic font, the structured task layout, and the hint system.

Each participant recorded how long they took to complete their lab and reported any technical problems they encountered. After finishing, they filled in a short questionnaire with four questions rated on a 1–5 scale (clarity of the guide, difficulty, overall experience, and whether the guide was enough to complete the lab), plus a few open questions about what was hardest and what they would change.

7.9.2 Quantitative Results

Table 7.9 shows the average scores and times for each lab.

Lab	*n*	Clarity	Difficulty	Experience	Avg. time (min)
Lab 1 — Reconnaissance	3	4.33	2.33	4.67	88
Lab 2 — Web Vulnerabilities	3	5.00	3.00	4.67	95
Lab 3 — Incident Response	3	4.00	3.67	4.67	122
Overall	9	**4.44**	**3.00**	**4.67**	**102**

Table 7.9: Pilot study results — mean scores (1–5 Likert) and resolution times

All five participants completed their labs. No technical problem stopped anyone from finishing. The average satisfaction score was **4.67 out of 5**, which is above the 4/5 target set in OE4 (Chapter 3). The average time across all labs was **102 minutes**, which is within the 90–120 minute target.

Lab 1 took an average of 88 minutes, just under the target. This is expected since it is the first lab and the most guided one. Lab 3 took an average of 122 minutes, just over the target. This makes sense because it includes a forensic analysis task and a written incident report, which take more time than a purely technical task. Both differences are small and fit the planned progression from easier to harder labs.

7.9.3 Qualitative Analysis

The open questions showed some common patterns across the three labs.

Getting started in the environment. Two participants had trouble at the beginning before the technical tasks even started. One spent around ten minutes trying to connect via SSH to the attacker machine in Lab 1, not knowing it only has a VNC console inside EVE-NG. Another participant in Lab 2 sent a payload to the wrong server because the guide did not explain upfront that nginx works as a reverse proxy in front of two backend servers. Both cases suggest that each student guide needs a short section at the beginning explaining how to access the lab and what the key parts of the topology are.

Understanding why, not just how. Several participants said they found it more helpful when the guide explained the reason behind a step, not just the command to run. In Lab 2, the choice of listener IP address was only explained in a late hint, so one participant did not understand it until that point. In Lab 3, using `find` to escalate privileges through GTFOBins[1] felt confusing without any context about what GTFOBins is. Adding a short explanation in the task body — not just in hints — would make the labs more educational.

Log readability. The participant with dyslexia found the `auth.log` output in Lab 3 very hard to read. Log lines with timestamps, hostnames, PIDs, and messages all together are a lot of information at once. The guide assumed students already knew how to use `grep` to filter the relevant lines. Adding a short introduction to log structure and showing one example of expected output for each `grep` command would help all students, not just those with reading difficulties.

Connecting the three labs. The participant who completed all three labs pointed out that the overall flow makes sense (reconnaissance,

[1] GTFOBins is a list of Unix binaries that can be used to bypass local security restrictions: https://gtfobins.github.io.

then web attack, then incident response) but felt that the connection between the labs could be stronger. Framing Lab 3 as the investigation of an incident caused by the same type of attack seen in Labs 1 and 2 would help students see the full picture — attack and defence as two sides of the same chain.

7.9.4 KPI Compliance

KPI	Target	Result	Status
Mean satisfaction score	$\geq 4/5$	4.67/5	✓
Lab completion rate	100%	9/9	✓
Mean resolution time	90–120 min	102 min	✓
Blocking technical incidents	0	0	✓

Table 7.10: OE4 KPI compliance after pilot study

All four KPIs from OE4 were met. The small differences in per-lab times are not a problem: the overall average is within target, and the pattern matches the intended difficulty progression of the lab sequence.

7.9.5 Identified Improvements

The pilot showed a number of small issues. None of them stopped anyone from finishing, but fixing them would make the labs clearer and more useful:

- **All labs**: add a short *Environment Access* section at the start of each student guide explaining console types (VNC vs. SSH) and the key parts of the topology.
- **Lab 2**: explain the listener IP choice in the task body instead of a hint; add a note about the nginx reverse proxy before the task sequence; remind students to check that the listener is active before posting the payload.
- **Lab 3**: add a note about boot time before the first SSH connection; clarify that the database server is only reachable from

inside the web server; add one line of expected `grep` output per command; add a short description of GTFOBins; add a reflection question at the end of the final task about real-world detection.

8 Conclusions

This chapter evaluates the project outcomes against the stated objectives, reflects on lessons learned during development, and outlines directions for future work. The overall goal — to design and implement a reusable, curriculum-aligned penetration testing lab package for the *Introduction to Cybersecurity* course at TecnoCampus — has been substantially achieved.

8.1 Summary of Achievements

The project delivered four progressive lab scenarios built on EVE-NG [1] and deployed on a self-hosted Proxmox [18] infrastructure:

- **Lab 1 — Reconnaissance and Enumeration**: A layered network topology featuring pfSense [39], VyOS [38], and an Ubuntu Server [40] target. Students perform passive and active reconnaissance using Nmap, DNS enumeration, and service fingerprinting, mirroring the PTES [10] initial access phase.
- **Lab 2 — Web Application Vulnerabilities (SYNAPSE)**: A custom two-tier web application aligned with OWASP Top 10 [11]. The exploitation chain — stored XSS, SQL injection, and command injection — guides students through a realistic attack scenario that requires lateral thinking and multi-step reasoning.
- **Lab 3 — Incident Response and Log Forensics (HELIX Systems)**: An inverted-role lab where students act as security analysts following NIST SP 800-61 [47]. Pre-staged log evidence and a reconstructed attack timeline develop forensic analysis skills alongside offensive concepts.
- **Lab 4 — Privilege Escalation**: Under development (Phase 3). The topology and scenario design are complete; implementation and validation will be finalised before the May 27, 2026 submission deadline.

Each lab is supported by a student guide, a teacher resolution guide with an assessment rubric, and automation scripts for deployment and environment reset. All documentation is published in three parallel layers: the formal LaTeX report (this volume), Volume II (Appendices), and the project web documentation site at theegea.github.io/TFG.

8.2 Objectives Evaluation

Table 8.1 summarises the four specific objectives defined in Chapter 3 and their current completion status.

Objective	Description	Status
OE1	Design and implement 4 EVE-NG labs	75% (3/4 completed)
OE2	Develop automation scripts	Completed
OE3	Develop structured teaching materials	75% (3/4 labs documented)
OE4	Validate usability and effectiveness	Completed

Table 8.1: Specific objectives evaluation at submission

OE1 is on track: Labs 1–3 are fully implemented and validated. Lab 4 design is complete; implementation is the main remaining activity in Phase 3. **OE2** is fully achieved: automation scripts cover ISO upload, bridge configuration, and connectivity checks, with at least one utility script per lab as required. **OE3** follows OE1 progress — three complete student and teacher guide pairs are available; Lab 4 materials will be produced in parallel with implementation. **OE4** is complete: the pilot study (May 2026) confirmed usability and flag reachability across all three implemented labs. KPI results — task completion, satisfaction scores, and KPI compliance — are reported in Section 7.9.

8.3 Lessons Learned

Infrastructure stability precedes content. The migration from VMware to Proxmox early in Phase 0 was the single most impactful

decision of the project. Developing lab content on an unstable hypervisor would have caused compounding delays. The lesson is general: validate the execution environment thoroughly before investing in content.

Image management is a project in itself. Selecting, importing, and maintaining QEMU images for EVE-NG required significantly more effort than expected. Naming conventions, permission scripts, and QCOW2 format validation each needed dedicated attention. The `iso_uploader.py` script emerged from this experience and reduced future onboarding time substantially.

Flag-based progression is effective for self-paced learning. The flag model — where students collect verifiable tokens at each milestone — provides immediate feedback and natural progress checkpoints without requiring teacher presence. The pilot study confirmed that students find the model motivating and that flags help them identify when they are on the wrong path.

Documentation as a first-class deliverable. Producing documentation in three layers (LaTeX, web, src/) added overhead but proved valuable: the web documentation became the primary reference during pilot sessions, while the LaTeX volume serves the formal academic record. Future projects of this type should plan the documentation architecture before starting implementation.

Time estimates for realistic EVE-NG scenarios are consistently optimistic. Each lab required approximately 20–30% more hours than initially estimated, mainly due to unexpected node configuration persistence issues and the need to validate flag reachability after each topology change. Any similar project plan should include an explicit buffer for integration testing.

8.4 Limitations and Future Work

8.4.1 Current Limitations

- **Lab 4 not yet complete:** Privilege escalation content is under development. The full four-lab package will be available upon submission.
- **No LMS integration:** The package is self-contained but not integrated with Moodle or any institutional learning management system. Assessment rubrics are PDF-based; submission and grading are manual.
- **Single-group concurrency:** The current infrastructure supports one lab group at a time. Running simultaneous sessions for multiple groups would require either additional server resources or a cloud-based EVE-NG deployment.

8.4.2 Directions for Future Work

- **Complete Lab 4:** Finalise the privilege escalation scenario with SUID, sudo misconfiguration, and cron-based persistence vectors; produce the student guide and teacher reference.
- **Expand OWASP coverage:** Lab 2 currently covers five Top 10 categories. Additional modules could address server-side request forgery (SSRF), insecure deserialization, and API security misconfigurations.
- **Moodle integration:** Export assessment rubrics as Moodle-compatible grading forms; implement automated flag submission via a simple REST endpoint.
- **Multi-group support:** Evaluate cloud EVE-NG deployment (AWS or on-premises scale-out) for concurrent execution by multiple student groups within the same course edition.
- **Lab 5 — Active Directory:** A Windows domain penetration testing scenario (Kerberoasting, Pass-the-Hash, lateral movement) to complement the existing Linux-centric labs and address OSCP-aligned skill gaps.

8.5 Personal Reflection

This project brought together skills from across the Computer Engineering degree — network administration, operating system internals, web security, scripting, and technical writing — within a single coherent artefact. Designing realistic attack scenarios, implementing them in a reproducible environment, and documenting them rigorously as teaching material was more challenging and more rewarding than expected.

Working within the constraints of academic infrastructure and a fixed timeline required constant prioritisation. The most durable decisions were those made at the architectural level early in Phase 0: choosing Proxmox over VMware, EVE-NG Community Edition over proprietary alternatives, and an open-source-only tool stack. These choices ensure that the package remains deployable and maintainable by any institution with comparable hardware, independent of licensing or vendor relationships.

This experience reinforced a conviction that practical, scenario-based learning is essential for cybersecurity education, and that well-designed lab environments can significantly narrow the gap between theoretical knowledge and professional practice. Delivering a package that will be used in a real course is the most meaningful validation of the work.

Bibliography

[1] "EVE-NG - emulated virtual environment next generation." Network simulation platform - primary platform for TFG. Accessed: 4 March 2026, EVE-NG Community. (2025), [Online]. Available: https://www.eve-ng.net/.

[2] Centre Universitari TecnoCampus. "Centre universitari tecnocampus — official website." Accessed: 2025, Centre Universitari TecnoCampus. (2024), [Online]. Available: https://www.tecnocampus.cat.

[3] "HackTheBox - cybersecurity training platform." Online pentesting platform with CTF challenges. Accessed: 4 March 2026, HackTheBox Ltd. (2024), [Online]. Available: https://www.hackthebox.com/.

[4] "TryHackMe - learn cybersecurity through hacking." Gamified cybersecurity training platform. Accessed: 4 March 2026, TryHackMe. (2024), [Online]. Available: https://tryhackme.com/.

[5] "SEED labs - hands-on lab exercises." Academic cybersecurity lab exercises. Accessed: 4 March 2026, SEED Project. (2024), [Online]. Available: https://seedsecuritylabs.org/.

[6] "GOAD - game of active directory." Vulnerable Active Directory lab environment. Accessed: 4 March 2026, GOAD Project. (2024), [Online]. Available: https://github.com/Orange-Cyberdefense/GOAD.

[7] "VulnHub - vulnerable machine community." Community-driven vulnerable virtual machines. Accessed: 4 March 2026, VulnHub Project. (2024), [Online]. Available: https://www.vulnhub.com/.

[8] "Hack4U - cybersecurity training platform." Spanish cybersecurity training platform with practical labs. Accessed: 4 March 2026, Hack4U. (2026), [Online]. Available: https://hack4u.io/.

[9] "Cisco networking academy." Official networking curriculum. Accessed: 4 March 2026, Cisco Systems Inc. (2024), [Online]. Available: https://www.netacad.com/.

[10] "Penetration testing execution standard (ptes)." Industry-standard pentesting framework. Accessed: 4 March 2026, PTES Executive Summary. (2014), [Online]. Available: http://www.pentest-standard.org/.

[11] "OWASP top 10 2021: A03:2021 – injection." Accessed: 5 January 2026, Open Web Application Security Project. (2021), [Online]. Available: https://owasp.org/Top10/.

[12] "CIS critical controls v8.0." Framework for defensive security. Accessed: 8 April 2026, Center for Internet Security. (2023), [Online]. Available: https://www.cisecurity.org/cis-controls/.

[13] "Framework for improving critical infrastructure cybersecurity." Version 1.1. Accessed: 8 April 2026, National Institute of Standards and Technology. (2022), [Online]. Available: https://www.nist.gov/cyberframework/framework.

[14] G. Conti and R. Di Pietro, "Hands-on cybersecurity: A challenge-based approach," *IEEE Security & Privacy*, vol. 17, no. 2, pp. 19–27, 2019. DOI: 10.1109/MSEC.2019.2891308.

[15] C. Murphy, R. Fitzpatrick, and D. Mac Cormac, "Design of laboratory exercises for cybersecurity curriculum," in *Proceedings of the 2018 IEEE Conference on Cyber Security and Data Protection (CSDP)*, 2018, pp. 1–8.

[16] M. Sponj and M. Heričko, "Cybersecurity education: A systematic review," *Journal of Educational Computing Research*, vol. 59, no. 4, pp. 528–562, 2021. DOI: 10.1177/0735633120968370.

[17] H. Al Mohanadi and S. Furnell, "A requirements-driven approach to designing an assessment laboratory for information security," *Journal of Educational Computing Research*, vol. 54, no. 4, pp. 469–494, 2016. DOI: 10.1177/0735633115623065.

[18] "Proxmox virtual environment." Open-source hypervisor and virtualization platform. Accessed: 4 March 2026, Proxmox Server Solutions. (2024), [Online]. Available: https://www.proxmox.com/.

[19] "Visual Studio Code - code editor." Primary code editor for development. Accessed: 4 March 2026, Microsoft Corporation. (2026), [Online]. Available: https://code.visualstudio.com/.

[20] "LaTeX - document preparation system." Typesetting system for technical documents. Accessed: 4 March 2026, LaTeX Project. (2026), [Online]. Available: https://www.latex-project.org/.

[21] "Parrot os - security distribution." Debian-based Linux distribution for security testing. Accessed: 8 April 2026, Parrot Security. (2024), [Online]. Available: https://www.parrotsec.org/.

[22] "Metasploit framework documentation." Exploitation framework. Accessed: 4 March 2026, Metasploit Project. (2025), [Online]. Available: https://docs.metasploit.com/.

[23] "Burp suite community edition." Web application security testing tool. Accessed: 4 March 2026, PortSwigger Web Security. (2024), [Online]. Available: https://portswigger.net/burp/communitydownload.

[24] "OWASP web security testing guide (wstg)." Version 4.2 - Comprehensive testing methodology. Accessed: 4 March 2026, Open Web Application Security Project. (2023), [Online]. Available: https://owasp.org/www-project-web-security-testing-guide/.

[25] "CEH certified ethical hacker exam objectives." Certification framework. Accessed: 4 March 2026, EC-Council. (2024), [Online]. Available: https://www.eccouncil.org/programs/certified-ethical-hacker-ceh/.

[26] "Technical security testing." Guidelines for security testing. Accessed: 8 April 2026, National Institute of Standards and Technology. (2008), [Online]. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf.

[27] "EVE-NG official documentation." Complete platform documentation. Accessed: 4 March 2026, EVE-NG Community. (2025), [Online]. Available: https://www.eve-ng.net/index.php/documentation.

[28] "OWASP ZAP - Web Application Security Scanner." Automated security scanning and testing. Accessed: 4 March 2026, Open Web Application Security Project. (2024), [Online]. Available: https://www.zaproxy.org/.

[29] "Nmap official documentation." Network discovery and security auditing tool. Accessed: 4 March 2026, Nmap Project. (2024), [Online]. Available: https://nmap.org/.

[30] "Wireshark - network protocol analyzer." Packet capture and network analysis tool. Accessed: 4 March 2026, Wireshark Foundation. (2024), [Online]. Available: https://www.wireshark.org/.

[31] "Grau en enginyeria informàtica de gestió i sistemes d'informació (GEISI)." Bachelor program curriculum and learning outcomes. Accessed: 4 March 2026, TecnoCampus - Universitat de Barcelona. (2024), [Online]. Available: https://www.tecnocampus.cat/grau/grau-en-enginyeria-informatica-de-gestio-i-sistemes-dinformacio.

[32] "Guide for conducting risk assessments." Accessed: 4 March 2026, National Institute of Standards and Technology. (2012), [Online]. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf.

[33] "Spanish electricity grid carbon intensity data." Official electricity generation and emissions data. Accessed: 4 March 2026, Red Eléctrica de España. (2024), [Online]. Available: https://www.ree.es/.

[34] S. Murugesan, "Harnessing green IT: Principles and practices," *IT Professional*, vol. 10, no. 1, pp. 24–33, 2008.

[35] "DVWA - damn vulnerable web application." Intentionally vulnerable web application for training. Accessed: 4 March 2026, DVWA Project. (2026), [Online]. Available: https://github.com/digininja/DVWA.

[36] (ISC)^2, *Cybersecurity workforce study 2023*, https://www.isc2.org/research/workforce-study, Accessed: April 2026, 2023.

[37] G. Conti and R. Di Pietro, "Hands-on cybersecurity: A challenge-based approach," *IEEE Security & Privacy*, vol. 17, no. 2, pp. 19–27, 2019. DOI: 10.1109/MSEC.2019.2891308.

[38] "VyOS - open source router and firewall." Open-source network OS for routing and firewalling. Accessed: 8 April 2026, VyOS Project. (2024), [Online]. Available: https://vyos.io/.

[39] "pfSense community edition." Open-source firewall and router platform based on FreeBSD. Accessed: 8 April 2026, Netgate. (2024), [Online]. Available: https://www.pfsense.org/.

[40] "Ubuntu server and desktop." Open-source Linux distribution. Version 24.04 LTS used for server and workstation nodes. Accessed: 8 April 2026, Canonical Ltd. (2024), [Online]. Available: https://ubuntu.com/.

[41] "Kali linux official documentation." Primary penetration testing distribution. Accessed: 4 March 2026, Kali Linux. (2025), [Online]. Available: https://www.kali.org/docs/.

[42] "VMware vsphere hypervisor (ESXi)." Enterprise hypervisor. Accessed: 4 March 2026, VMware Inc. (2024), [Online]. Available: https://www.vmware.com/products/esxi-and-esx.html.

[43] P. Mockapetris, *RFC 1035: Domain Names — Implementation and Specification*, IETF, 1987. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1035.

[44] J. Postel, *RFC 793: Transmission Control Protocol*, IETF, 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc793.

[45] T. Ylonen and C. Lonvick, *RFC 4252: The Secure Shell (SSH) Authentication Protocol*, IETF, 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4252.

[46] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, *RFC 1918: Address Allocation for Private Internets*, IETF, 1996. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1918.

[47] "Computer security incident handling guide." Primary NIST guide for incident response. Accessed: 27 April 2026, National Institute of Standards and Technology. (2012), [Online]. Available: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf.